

Connectivity & Integration Bugs										
ID	Test Case Name	Under the hood Reason	How to Reproduce							
CON-001	Date format mismatches in Profile Forms	Frontend datepickers serialize dates as ISO 8601 strings, while backend/Prisma expects date-only (YYYY-MM-DD) or timestamps, resulting in 400 Bad Request or timezone-shifting errors.	Edit experiences/education and save. Inspect the network payload to /api/profile/experiences. Check if date values shift by +/- 1 day due to timezone translation.	No	Pages	Functionalities	Is Frontend Sync with Backend	Is Working on Dev Environment		
CON-002	Hardcoded Port Rewrites bypassing Env configuration	Service ports (4001, 4002, 4003, etc.) are hardcoded in next.config.js rewrites, which will fail in Docker or production environments when ports/hosts are dynamically mapped.	Run the UI in production mode pointing to a staging or containerized API. Check if API routes (e.g. /api/auth/*) throw 502 Bad Gateway or 404 Not Found because backend services are at different hosts/ports.	1	/settings (Seeker & Employer)	Connected Accounts & Password Change	No	No		Authentication method check is flawed (shows Google OAuth incorrectly for email/password users). Password change logic is missing or broken.
CON-003	CORS Blockage on Direct Sub-Component API Calls	Direct API requests bypassing Next.js server-side rewrites (e.g., direct socket connections or browser-to-service calls) fail due to missing microservice CORS policies.	Inspect the browser's DevTools Console. Look for "Access to fetch at..." has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present.	2	/settings (Seeker & Employer)	Account Info, Preferences, Active Sessions, Billing	Partially	Yes (UI only)		Form state uses dummy arrays (e.g., Active Sessions, billing data). Needs integration with proper backend endpoints for all sections.
CON-004	Hangs/Crashes on Expired JWT Access Tokens	JWT expirations are managed by the Auth microservice (Port 4001), if the frontend fails to implement silent token refreshes via cookies or /api/auth/refresh, profile updates will fail with 401.	Log in, wait for JWT access token expiry (or manually delete/edit the local JWT storage), and try to update skills or projects. Verify if the UI hangs/crashes instead of redirecting to login.	3	/employer/applicants & [id]	Applicant Details, Status Update, Resume Download, Schedule Interview	No	Yes (UI only)		Action buttons (Download Resume, Message, Save Note either do nothing or only update local React state. Real backend hooks are missing.
CON-005	ID Type Mismatches (UUID vs Auto-Incrementing Int)	Prisma schemas may use auto-incrementing integer IDs (Int) for records, whereas the frontend Next.js routes and dynamic URLs assume string UUIDs (String).	Navigate to a detail page (e.g., /profile/projects/[id]). Check if routing a numeric ID throws a 404 or causes NestJS backend errors due to incorrect database ID parsing.	4	/messages & employer/messages	Real-time messaging engine	No	Yes (UI only)		Completely mocked using localStorage (cykruit_messages) and static mock files (@lib/messages-data.ts). Requires a real backend (e.g., WebSockets).
CON-006	Hydration Errors on Page Failback States	Next.js server-side rendering (SSR) mismatch where the server compiles mock html but client loads fresh backend data, causing layout shift or hydration crashes.	Load a profile page containing backend-populated data directly via browser refresh. Look for console warnings: "Text content did not match. Server: "... Client: "...".	5	/dashboard & employer/dashboard	Dashboard Summary, Recent Applications, Stats	partially	Yes (UI only)		Relies heavily on static arrays (RECENT_APPS, RECOMMENDED_JOBS) or localStorage. Failbacks trigger when the API fails to fetch.
CON-007	WebSocket Handshake Authentication Failure (/messages)	The real-time messaging gateway on NestJS requires a JWT in the connection query parameters or headers during the handshake, but the frontend hooks up to ws:// without passing auth tokens.	Navigate to the Messages tab. Check if the connection state remains disconnected and search for connection error codes (e.g., 401 Unauthorized / 1011 Server Error) in the Network -> WS tab.	6	/profile (Seeker)	Profile Editing (Bio, Skills, Experience, etc.)	Yes (but failing)	No		The UI exists and attempts to patch the profile, but the save operation fails (likely a missing or broken backend endpoint).
CON-008	KYC Document Upload Payload Limitations (kyc)	Frontend limits file inputs to a higher boundary (e.g., 20MB), but NestJS body-parser defaults to 1MB or 100KB limits, causing request body overflow blockages.	Attempt to upload a high-resolution KYC verification document (e.g., SMB PDF or image). Verify if the API returns 413 Payload Too Large.	7	/employer/company & notifications	Company Profile & Notifications List	No	Yes (UI only)		Company profile state has no fetch/save hooks. Notifications rely entirely on localStorage (cykruit_notifications).
Hardcoded UI/UX				8	Employer/Sidebar.tx (Component)	Sidebar Active State Highlighting	NA	No		Logic bug: pathname.startsWith(href) causes both /employer/jobs and /employer/jobs/new to highlight simultaneously.
				9	/employer/jobs	Fetching employer jobs list	Yes (but failing)	No		The frontend tries to fetch /api/employer/jobs but encounters an error (possibly a 404 or 500 from the backend), showing "Failed to fetch jobs".
				10	/saved (Seeker)	Saving and browsing bookmarked jobs	no	Yes (UI only)		Relies entirely on localStorage (cykruit_saved_jobs) instead of persisting the saved jobs to a user database.
ID	Test Case Name	Under the hood Reason	How to Reproduce	11	/employer/jobs/new	"AI Suggest Skills" & "AI Generate Questions"	no	Yes (UI only)		Features are simulated using setTimeout and hardcoded static arrays rather than calling an actual AI backend endpoint.
HAR-001	Static Mock Leaderboard / CTF Achievements	Frontend UI mockups present pre-populated fake data (e.g., "Rank #1", "Score: 9999") because the integration with the public CTF service (Port 4006) is missing.	Register a new user and open the dashboard/leaderboard. Check if the scoreboard shows mock data instead of dynamic database rows or a clean empty-state.	12	/employer/subscription	Subscription Plan & Invoices Management	no	Yes (UI only)		Entirely hardcoded with dummy plan details and payment methods. No integration with Stripe or any payment processor.
HAR-002	Static Mock Jobs on Employer Panel	The employer UI shows dummy job listings hardcoded within page components instead of making fetch requests to the Employer service (Port 4004).	Log in as an Employer. Try to create or edit a job listing. Check if changes persist upon page reload. If listings remain identical regardless of database contents, they are hardcoded.	13	/applications (Seeker)	Job Applications Tracking	partially	Yes (UI only)		Fallback mock data triggers if the api/seeker/applications fetch fails. Lacks pagination and real-time updates.
HAR-003	Hardcoded User Profile Data in Layout Headers	Layouts (e.g. Navbars, SideLists) use hardcoded placeholder values (e.g. "John Doe", "john.doe@example.com") instead of pulling the auth session state.	Log in as a user with a unique name/email. Check if the header still displays default mock text. Edit the profile and verify if the header immediately updates.							
HAR-004	Hardcoded Resume/Media Download Links	The "Download Resume" button links to a static PDF asset in the frontend's/public folder rather than pulling dynamic database URLs.	Upload a custom resume PDF, click download, and verify if the downloaded file is the one you uploaded or a generic placeholder PDF.							
HAR-005	Static System Error Notifications	Frontend displays static, generic errors (e.g. "Something went wrong!") instead of parsing backend error responses (e.g. "Username already taken").	Trigger a known database constraint failure (e.g., sign up with an existing email). Check if the UI displays the specific validation warning or a generic error.							
HAR-006	Mock Stripe / Subscription Checkout Plans (pricing)	The frontend pricing options have static plans and amounts that do not reference or sync with current backend Stripe product IDs / price keys.	Click "Subscribe" on a pricing plan. Verify if the gateway fails, checks out with the wrong price ID, or triggers a Stripe API mismatch error (No such price: price_...).							
UX and Flaw Breaks										
ID	Test Case Name	Under the hood Reason	How to Reproduce							
FLW-001	Silent Failures on Form Submission	form catch blocks or fetch request logic omit user-facing alerts/toasts, failing silently when the backend service is offline.	Temporarily stop the backend Profile service (localhost:4003), submit a skill or experience update, and check if the form closes or spins indefinitely without error messages.							
FLW-002	Infinite Loading Spinners on Non-200 Responses	The React state variable loading is set to true when initiating a fetch request, but is never toggled to false in the exception block.	Send a request that returns a 404, 401, or 500 error. Check if the UI remains locked in an overlay loader or skeleton state indefinitely.							
FLW-003	Stale UI State / Lack of Data Revalidation	Missing state revalidation triggers (e.g., using SWR/React Query mutate, or Next.js router.refresh) after successful POST/PUT operations.	Add an item (e.g., education, certificate). Click save. If the item doesn't appear in the list until you manually refresh the page, state revalidation is missing.							
FLW-004	Input Overflow & Missing Form Constraints	Frontend textareas and inputs lack maxLength limits, allowing data entries that exceed Prisma database schemas, causing DB crashes.	Paste a very long text string (e.g., 5000+ characters) into a profile field and save. Check if the backend crashes with a database error, or if the layout breaks on rendering.							
FLW-005	Unsaved Changes Discard Alert	React form states do not monitor the input dirty flags or hook into route-blocking/beforeunload events to warn users before losing input data.	Type text in a detailed form (like adding a project). Click a link in the navigation menu or the cancel button. Verify if the app discards changes without warning.							
FLW-006	Absolute Port Verification Email Links (Verify-email)	Backend emailing constructs the verification link using its internal port URL (e.g., http://localhost:4001/verify...) instead of directing the user back to the Next.js port.	Trigger a registration and check the verification email link. Confirm that clicking the link points to the frontend verification landing page (http://localhost:3000/) rather than a backend service port.							